

Projeto SGBD ITP

Gerado por Doxygen 1.9.8

1 SGBD ITP	1
1.1 Instalação	1
1.2 Detalhes da implementação	2
1.3 O que poderia ser feito de forma diferente?	2
1.4 Autora	2
2 Índice das Estruturas de Dados	3
2.1 Estruturas de Dados	3
3 Índice dos Arquivos	5
3.1 Lista de Arquivos	5
4 Estruturas	7
4.1 Referência da Estrutura ColunaDaTabela	7
4.1.1 Descrição detalhada	7
4.1.2 Campos	7
4.1.2.1 dados	7
4.1.2.2 nome	7
4.1.2.3 quantidadeDeLinhas	7
4.1.2.4 tipo	7
4.2 Referência da Estrutura Tabela	8
4.2.1 Descrição detalhada	8
4.2.2 Campos	8
4.2.2.1 colunas	8
4.2.2.2 nome	8
4.2.2.3 quantidadeDeColunas	8
4.2.2.4 quantidadeDeLinhas	8
4.3 Referência da Estrutura Texto	8
4.3.1 Descrição detalhada	8
4.3.2 Campos	9
4.3.2.1 capacidade	9
4.3.2.2 tamanho	9
4.3.2.3 vetor	9
5 Arquivos	11
5.1 Referência do Arquivo Dados.c	11
5.1.1 Funções	11
5.1.1.1 ehDigito()	11
5.1.1.2 ehMaiuscula()	11
5.1.1.3 ehMinuscula()	12
5.1.1.4 ehSimbolo()	12
5.1.1.5 texto_gravar()	12
5.1.1.6 texto_lerTerminalEmLoop()	12
5.1.1.7 texto_lerTipoEmLoop()	13

5.1.1.8 texto_lerValorComTipo()	13
5.1.1.9 texto_lerValorDoTipoString()	13
5.1.1.10 texto_lerValorPositivoEmLoop()	13
5.1.1.11 texto_recuperar()	14
5.1.1.12 verificaNomeDaTabelaValido()	14
5.2 Referência do Arquivo Dados.h	14
5.2.1 Funções	15
5.2.1.1 texto_gravar()	15
5.2.1.2 texto_lerTerminalEmLoop()	15
5.2.1.3 texto_lerTipoEmLoop()	15
5.2.1.4 texto_lerValorComTipo()	15
5.2.1.5 texto_lerValorPositivoEmLoop()	16
5.2.1.6 texto_recuperar()	16
5.3 Dados.h	16
5.4 Referência do Arquivo Pesquisa.c	17
5.4.1 Enumerações	17
5.4.1.1 Regra	17
5.4.2 Funções	17
5.4.2.1 compararProximo()	17
5.4.2.2 compararValores()	18
5.4.2.3 deveExibir()	18
5.4.2.4 pesquisa_exibir()	18
5.4.2.5 pesquisar()	19
5.5 Referência do Arquivo Pesquisa.h	19
5.5.1 Funções	19
5.5.1.1 pesquisar()	19
5.6 Pesquisa.h	19
5.7 Referência do Arquivo principal.c	20
5.7.1 Enumerações	20
5.7.1.1 Opcoes	20
5.7.2 Funções	21
5.7.2.1 adicionarDados()	21
5.7.2.2 apagarTabela()	21
5.7.2.3 apagarTupla()	21
5.7.2.4 criarTabela()	21
5.7.2.5 exibirDados()	21
5.7.2.6 leituraDaChavePrimaria()	21
5.7.2.7 leituraDeDados()	21
5.7.2.8 main()	22
5.7.2.9 menu()	22
5.7.2.10 pesquisarValores()	22
5.7.2.11 verificarExistenciaDaChave()	22

5.7.2.12 verificarNomesIguais()	22
5.8 Referência do Arquivo README.md	23
5.9 Referência do Arquivo Tabela.c	23
5.9.1 Funções	23
5.9.1.1 coluna_buscarValor()	23
5.9.1.2 tabela_adicionarLinha()	23
5.9.1.3 tabela_criar()	24
5.9.1.4 tabela_exibirCabecalho()	24
5.9.1.5 tabela_exibirDados()	24
5.9.1.6 tabela_gravar()	24
5.9.1.7 tabela_liberar()	25
5.9.1.8 tabela_recuperar()	25
5.9.1.9 tabela_removeChave()	25
5.10 Referência do Arquivo Tabela.h	26
5.10.1 Funções	26
5.10.1.1 coluna_buscarValor()	26
5.10.1.2 tabela_adicionarLinha()	26
5.10.1.3 tabela_criar()	27
5.10.1.4 tabela_exibirCabecalho()	27
5.10.1.5 tabela_exibirDados()	27
5.10.1.6 tabela_gravar()	28
5.10.1.7 tabela_liberar()	28
5.10.1.8 tabela_recuperar()	28
5.10.1.9 tabela_removeChave()	28
5.11 Tabela.h	29
5.12 Referência do Arquivo TabelaGeral.c	29
5.12.1 Definições e macros	30
5.12.1.1 NOME_DA_TABELA_GERAL	30
5.12.2 Funções	30
5.12.2.1 tabelaGeral_adicionarTabela()	30
5.12.2.2 tabelaGeral_apagar()	30
5.12.2.3 tabelaGeral_contemTabela()	30
5.12.2.4 tabelaGeral_gravarDados()	30
5.12.2.5 tabelaGeral_limpar()	31
5.12.2.6 tabelaGeral_listar()	31
5.12.2.7 tabelaGeral_recuperarDados()	31
5.13 Referência do Arquivo TabelaGeral.h	31
5.13.1 Funções	32
5.13.1.1 tabelaGeral_adicionarTabela()	32
5.13.1.2 tabelaGeral_apagar()	32
5.13.1.3 tabelaGeral_contemTabela()	32
5.13.1.4 tabelaGeral_limpar()	32

5.13.1.5 tabelaGeral_listar()	32
5.14 TabelaGeral.h	33
Índice Remissivo	35

Capítulo 1

SGBD ITP

O projeto consiste em implementar um Sistema de Gerenciamento de Banco de Dados (SGBD) simplificado baseado no modelo relacional. Não é o objetivo desse projeto criar um SGBD para ser utilizado em produção por sistemas de informação, mas “apenas” explorar os conceitos abordados na disciplina ITP. O SGBD ITP deverá ser capaz de realizar as seguintes operações:

1. Criar uma tabela;
2. Listar todas as tabelas;
3. Criar uma nova linha na tabela;
4. Listar todos os dados da tabela;
5. Pesquisar valor em uma tabela;
6. Apagar uma tupla ou linha de uma tabela;
7. Apagar uma tabela;
8. Apagar todas as tabelas (funcionalidade adicional).

O SGBD ITP implementado cria um arquivo para cada tabela, além de criar um arquivo extra contendo a quantidade de tabelas e seus nomes para auxiliar, quando necessário, na implementação das funções.

O arquivo que contém a quantidade de tabelas e seus nomes se chama **Tabela Geral.data** e está organizado da seguinte forma: quantidade de tabelas, nomes das tabelas.

Cada arquivo das tabelas está organizado da seguinte forma: nome da tabela, quantidade de colunas, quantidade de linhas, tipo de dado e nome da coluna intercalados, dados das colunas.

O código fonte foi dividido em 4 módulos além do principal, chamados: **Dados.h**, **Pesquisa.h**, **Tabela.h**, **TabelaGeral.h** e **principal.c**. No módulo **Dados**, estão organizadas as funções responsáveis pelas leituras de dados e gravações das informações obtidas nos arquivos. No módulo **Pesquisa**, estão organizadas as funções responsáveis pela pesquisa de um valor em uma tabela. No módulo **Tabela**, estão organizadas as funções responsáveis pelas operações relacionadas às tabelas, como a criação de uma tabela, a adição ou remoção de uma linha, a exibição de seus dados... No módulo **TabelaGeral**, estão organizadas as funções responsáveis pelas operações relacionadas ao arquivo 'tabela geral.data', como sua criação, a atualização de suas informações, a exibição dos nomes de todas as tabelas, a remoção de um arquivo... No módulo principal, está implementada a função principal, além de outras auxiliares.

1.1 Instalação

O código fonte está escrito na linguagem C, sendo necessário apenas de um compilador para a referida linguagem voltado a sua plataforma. Por exemplo, para o compilador GCC, pode executar a seguinte linha de comando no diretório que se encontra o código fonte:

```
gcc -o main -Wno-unused-result principal.c Dados.c Pesquisa.c Tabela.c TabelaGeral.c
```

Versão do compilador GCC: gcc version 6.3.0 (MinGW.org GCC-6.3.0-1) Link para download: <https://www.mingw-w64.org/> Importante ressaltar que não é necessária a instalação de nenhuma biblioteca externa.

1.2 Detalhes da implementação

Todas as funções solicitadas foram implementadas, além de uma função extra (Apagar todas as tabelas do código fonte) e a opção de sair do menu.

1.3 O que poderia ser feito de forma diferente?

Poderia ter sido criado um único arquivo para todas as tabelas. Porém, a implementação se tornou muito complexa para a autora, que decidiu reiniciar o projeto utilizando a ideia atual. Outra coisa que poderia ter sido diferente era ter salvo os dados nos arquivos de modo binário e não em caractere, reduzindo assim o tamanho do arquivo. Também poderia ter sido implementada uma função extra de remover uma coluna na tabela.

1.4 Autora

A autora do projeto se chama Camila Braga de Oliveira.

Capítulo 2

Índice das Estruturas de Dados

2.1 Estruturas de Dados

Aqui estão as estruturas de dados, uniões e suas respectivas descrições:

ColunaDaTabela	7
Tabela	8
Texto	8

Capítulo 3

Índice dos Arquivos

3.1 Lista de Arquivos

Esta é a lista de todos os arquivos e suas respectivas descrições:

Dados.c	11
Dados.h	14
Pesquisa.c	17
Pesquisa.h	19
principal.c	20
Tabela.c	23
Tabela.h	26
TabelaGeral.c	29
TabelaGeral.h	31

Capítulo 4

Estruturas

4.1 Referência da Estrutura ColunaDaTabela

```
#include <Tabela.h>
```

Campos de Dados

- [Texto nome](#)
- char [tipo](#)
- int [quantidadeDeLinhas](#)
- [Texto](#) * [dados](#)

4.1.1 Descrição detalhada

Representa uma coluna de uma tabela.

Representa uma coluna de uma tabela. Uma coluna tem um nome, que é representado pela estrutura de [Texto](#), um tipo associado, que pode ser 'i', 'f', 'd', 'c' ou 's', a quantidade de linhas presentes na coluna e um vetor para armazenar os dados da coluna.

4.1.2 Campos

4.1.2.1 dados

```
Texto* dados
```

Os dados da coluna

4.1.2.2 nome

```
Texto nome
```

Nome da coluna

4.1.2.3 quantidadeDeLinhas

```
int quantidadeDeLinhas
```

quantidade de linhas.

4.1.2.4 tipo

```
char tipo
```

o tipo de dado associado à coluna.

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- [Tabela.h](#)

4.2 Referência da Estrutura Tabela

```
#include <Tabela.h>
```

Campos de Dados

- [Texto](#) nome
- int [quantidadeDeColunas](#)
- int [quantidadeDeLinhas](#)
- [ColunaDaTabela](#) * [colunas](#)

4.2.1 Descrição detalhada

Representa uma tabela.

Representa uma tabela de dados do sistema. Uma tabela possui um nome, que é representado pela estrutura de [Texto](#), um vetor para armazenar os dados de cada uma das colunas e uma variável para indicar a quantidade de colunas presentes na tabela. Os detalhes de cada coluna serão armazenados pela estrutura [ColunaDaTabela](#). Além disso, possui um campo para indicar a quantidade linhas das tabelas, ou seja, a quantidade de dados em cada coluna.

4.2.2 Campos

4.2.2.1 colunas

[ColunaDaTabela](#)* [colunas](#)
dados das colunas da tabela

4.2.2.2 nome

[Texto](#) nome
Nome da tabela

4.2.2.3 quantidadeDeColunas

int [quantidadeDeColunas](#)
Quantidade de colunas da tabela

4.2.2.4 quantidadeDeLinhas

int [quantidadeDeLinhas](#)
Quantidade de linhas da tabela

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- [Tabela.h](#)

4.3 Referência da Estrutura Texto

```
#include <Dados.h>
```

Campos de Dados

- int [capacidade](#)
- int [tamanho](#)
- char * [vetor](#)

4.3.1 Descrição detalhada

Representa uma string de caracteres. Tem como objetivo armazenar informações de uma string, como seu tamanho, para facilitar as manipulações com arquivos.

4.3.2 Campos

4.3.2.1 capacidade

`int capacidade`

quantidade de posições alocadas para o vetor

4.3.2.2 tamanho

`int tamanho`

tamanho do texto sem o `\0`

4.3.2.3 vetor

`char* vetor`

vetor de caracteres do texto

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- [Dados.h](#)

Capítulo 5

Arquivos

5.1 Referência do Arquivo Dados.c

```
#include <string.h>
#include "Dados.h"
```

Funções

- int [ehMinuscula](#) (char c)
- int [ehMaiuscula](#) (char c)
- int [ehDigito](#) (char c)
- int [ehSimbolo](#) (char c)
- int [verificaNomeDaTabelaValido](#) (char *nome, int tamanho)
- void [texto_gravar](#) ([Texto](#) texto, FILE *arquivo)
- [Texto](#) [texto_recuperar](#) (FILE *arquivo)
- [Texto](#) [texto_lerTerminalEmLoop](#) (int capacidade)
- int [texto_lerValorPositivoEmLoop](#) ()
- char [texto_lerTipoEmLoop](#) ()
- [Texto](#) [texto_lerValorDoTipoString](#) ()
- [Texto](#) [texto_lerValorComTipo](#) (char tipo)

5.1.1 Funções

5.1.1.1 ehDigito()

```
int ehDigito (
    char c )
```

Verifica se o caractere é um dígito.

Parâmetros

c	Caractere sendo verificado.
---	-----------------------------

Retorna

1 se o caractere for um dígito e 0 em caso contrário.

5.1.1.2 ehMaiuscula()

```
int ehMaiuscula (
    char c )
```

Verifica se o caractere é uma letra maiúscula.

Parâmetros

<i>c</i>	Caractere sendo verificado.
----------	-----------------------------

Retorna

1 se o caractere for uma letra maiúscula e 0 em caso contrário.

5.1.1.3 ehMinuscula()

```
int ehMinuscula (
    char c )
```

Verifica se o caractere é uma letra minúscula.

Parâmetros

<i>c</i>	Caractere sendo verificado.
----------	-----------------------------

Retorna

1 se o caractere for uma letra minúscula e 0 em caso contrário.

5.1.1.4 ehSimbolo()

```
int ehSimbolo (
    char c )
```

Verifica se o caractere é um símbolo.

Parâmetros

<i>c</i>	Caractere sendo verificado.
----------	-----------------------------

Retorna

1 se o caractere for um símbolo e 0 em caso contrário.

5.1.1.5 texto_gravar()

```
void texto_gravar (
    Texto texto,
    FILE * arquivo )
```

Grava os dados do texto em um arquivo aberto em modo de escrita de caracteres.

Os dados são salvos em dois campos no seguinte formato: "%d %s\n", correspondendo ao tamanho do texto e ao texto em si.

Parâmetros

<i>texto</i>	texto a ser gravado.
<i>arquivo</i>	Arquivo onde serão gravados os dados. Ele deve estar aberto em modo de escrita de caracteres.

5.1.1.6 texto_lerTerminalEmLoop()

```
Texto texto_lerTerminalEmLoop (
    int capacidade )
```

Faz a leitura de um texto e verifica se ele é válido. Caso seja válido, retorna uma estrutura com os dados lidos e, em caso contrário, solicita ao usuário que digite outro texto.

Para ser válido, o texto digitado deve conter apenas letras, números, ' ', '_', '-' ou '.' além de iniciar apenas por letra ou dígito.

Parâmetros

<i>capacidade</i>	Espaço do vetor para leitura dos dados.
-------------------	---

Retorna

Uma estrutura de texto com os dados lidos.

5.1.1.7 texto_lerTipoEmLoop()

```
char texto_lerTipoEmLoop ( )
```

Faz a leitura de um caractere e verifica se o dado digitado é válido. Para ser válido, o caractere deve ser apenas 'i', 'f', 'd', 'c' ou 's'. Em caso de sucesso, retorna o dado. Caso contrário, solicita ao usuário um dado novo.

Retorna

Um caractere contendo o dado lido.

5.1.1.8 texto_lerValorComTipo()

```
Texto texto_lerValorComTipo (
    char tipo )
```

Faz a leitura e conversão de dados de acordo com a opção escolhida pelo usuário.

Recebe como argumento o tipo de dado da coluna escolhida pelo usuário, faz a leitura dos dados, os converte em string e depois armazena os dados convertidos no vetor de string. Observação: Para a variável "tipo" igual a 'f' ou 'd', a leitura e tratamento serão as mesmas.

Parâmetros

<i>tipo</i>	Tipo de dado da coluna escolhida previamente pelo usuário.
-------------	--

Retorna

Uma estrutura de texto com os dados lidos.

5.1.1.9 texto_lerValorDoTipoString()

```
Texto texto_lerValorDoTipoString ( )
```

Faz a leitura e tratamento de um texto, retornando uma estrutura com os dados desse texto.

Realiza a leitura caractere a caractere, enquanto não for digitado um '\n' ou um '\0'. Caso seja digitado '\n', realiza o tratamento que consiste em sobrepor esse último caractere digitado por '\0'.

Retorna

Uma estrutura de texto com os dados lidos.

5.1.1.10 texto_lerValorPositivoEmLoop()

```
int texto_lerValorPositivoEmLoop ( )
```

Realiza a leitura de um inteiro positivo e verifica se é um dado válido. Caso seja válido, retorna o dado lido e, em caso contrário, solicita ao usuário um novo dado.

Para ser válido, o número deve ser um inteiro não negativo.

Retorna

Um inteiro contendo o dado lido.

5.1.1.11 texto_recuperar()

```
Texto texto_recuperar (
    FILE * arquivo )
```

Recupera os dados de um texto de um arquivo aberto em modo de leitura de caracteres. Assume que os dados foram salvos no formato da função

Veja também

[texto_gravar](#).

Parâmetros

<i>arquivo</i>	Arquivo que será lido. Deve estar aberto no formado de leitura de caracteres.
----------------	---

Retorna

Uma estrutura de [Texto](#) com os dados lidos.

5.1.1.12 verificaNomeDaTabelaValido()

```
int verificaNomeDaTabelaValido (
    char * nome,
    int tamanho )
```

Verifica se um nome é válido.

Os nomes das tabelas possuem restrições quanto aos caracteres. Para serem válidos, os nomes devem ser compostos por letras, dígitos, símbolos '-', '_', '.' ou espaço, além de não poderem ser vazios nem começarem por espaço ou símbolo.

Parâmetros

<i>nome</i>	Ponteiro para nome a ser verificado.
<i>tamanho</i>	Quantidade de caracteres do nome.

Retorna

1 caso o nome seja válido e 0 em caso contrário.

5.2 Referência do Arquivo Dados.h

```
#include <stdio.h>
#include <stdlib.h>
```

Estruturas de Dados

- struct [Texto](#)

Funções

- void [texto_gravar](#) ([Texto](#) texto, FILE *arquivo)
- [Texto](#) [texto_recuperar](#) (FILE *arquivo)

- Texto `texto_lerTerminalEmLoop` (int capacidade)
- char `texto_lerTipoEmLoop` ()
- int `texto_lerValorPositivoEmLoop` ()
- Texto `texto_lerValorComTipo` (char tipo)

5.2.1 Funções

5.2.1.1 `texto_gravar()`

```
void texto_gravar (
    Texto texto,
    FILE * arquivo )
```

Grava os dados do texto em um arquivo aberto em modo de escrita de caracteres.

Os dados são salvos em dois campos no seguinte formato: "%d %s\n", correspondendo ao tamanho do texto e ao texto em si.

Parâmetros

<i>texto</i>	texto a ser gravado.
<i>arquivo</i>	Arquivo onde serão gravados os dados. Ele deve estar aberto em modo de escrita de caracteres.

5.2.1.2 `texto_lerTerminalEmLoop()`

```
Texto texto_lerTerminalEmLoop (
    int capacidade )
```

Faz a leitura de um texto e verifica se ele é válido. Caso seja válido, retorna uma estrutura com os dados lidos e, em caso contrário, solicita ao usuário que digite outro texto.

Para ser válido, o texto digitado deve conter apenas letras, numeros, '.', '_', '-' ou ':' além de iniciar apenas por letra ou dígito.

Parâmetros

<i>capacidade</i>	Espaço do vetor para leitura dos dados.
-------------------	---

Retorna

Uma estrutura de texto com os dados lidos.

5.2.1.3 `texto_lerTipoEmLoop()`

```
char texto_lerTipoEmLoop ( )
```

Faz a leitura de um caractere e verifica se o dado digitado é válido. Para ser válido, o caractere deve ser apenas 'i', 'f', 'd', 'c' ou 's'. Em caso de sucesso, retorna o dado. Caso contrário, solicita ao usuário um dado novo.

Retorna

Um caractere contendo o dado lido.

5.2.1.4 `texto_lerValorComTipo()`

```
Texto texto_lerValorComTipo (
    char tipo )
```

Faz a leitura e conversão de dados de acordo com a opção escolhida pelo usuário.

Recebe como argumento o tipo de dado da coluna escolhida pelo usuário, faz a leitura dos dados, os converte em string e depois armazena os dados convertidos no vetor de string. Observação: Para a variável "tipo" igual a 'f' ou 'd', a leitura e tratamento serão as mesmas.

Parâmetros

<i>tipo</i>	Tipo de dado da coluna escolhida previamente pelo usuário.
-------------	--

Retorna

Uma estrutura de texto com os dados lidos.

5.2.1.5 texto_lerValorPositivoEmLoop()

```
int texto_lerValorPositivoEmLoop ( )
```

Realiza a leitura de um inteiro positivo e verifica se é um dado válido. Caso seja válido, retorna o dado lido e, em caso contrário, solicita ao usuário um novo dado.

Para ser válido, o número deve ser um inteiro não negativo.

Retorna

Um inteiro contendo o dado lido.

5.2.1.6 texto_recuperar()

```
Texto texto_recuperar (
    FILE * arquivo )
```

Recupera os dados de um texto de um arquivo aberto em modo de leitura de caracteres.

Assume que os dados foram salvos no formato da função

Veja também

[texto_gravar](#).

Parâmetros

<i>arquivo</i>	Arquivo que será lido. Deve estar aberto no formato de leitura de caracteres.
----------------	---

Retorna

Uma estrutura de [Texto](#) com os dados lidos.

5.3 Dados.h

[Ir para a documentação desse arquivo.](#)

```
00001
00002 #ifndef DADOS_H
00003 #define DADOS_H
00004
00005 #include <stdio.h>
00006 #include <stdlib.h>
00007
00012 typedef struct
00013 {
00014     int capacidade;
00015     int tamanho;
00016     char *vetor;
00017 } Texto;
00018
00019 void texto_gravar(Texto texto, FILE *arquivo);
00020 Texto texto_recuperar(FILE *arquivo);
00021 Texto texto_lerTerminalEmLoop(int capacidade);
00022 char texto_lerTipoEmLoop();
00023 int texto_lerValorPositivoEmLoop();
00024
00025 // NULL para dado faltando
00026 Texto texto_lerValorComTipo(char tipo);
00027
00028 #endif
```

5.4 Referência do Arquivo Pesquisa.c

```
#include "Dados.h"
#include "Tabela.h"
#include "Pesquisa.h"
```

Enumerações

- enum `Regra` {
 `MAIOR` = 1 , `MAIOR_OU_IGUAL` = 2 , `IGUAL` = 3 , `MENOR_OU_IGUAL` = 4 ,
 `MENOR` = 5 , `PROXIMO` = 6 }

Funções

- int `compararValores` (`Texto` primeiro, `Texto` segundo, char tipo)
- int `compararProximo` (`Texto` primeiro, `Texto` segundo)
- int `deveExibir` (`Texto` valor, `Texto` comparar, `Regra` regra, char tipo)
- void `pesquisa_exibir` (`Tabela` tabela, int indice, `Texto` valor, `Regra` regra)
- void `pesquisar` (`Tabela` tabela, int indice)

5.4.1 Enumerações

5.4.1.1 Regra

enum `Regra`

Representa as opções de escolha do usuário referente à pesquisa de valores.

Enumeradores

<code>MAIOR</code>	Para valores maiores do que o solicitado;
<code>MAIOR_OU_IGUAL</code>	Para valores maiores ou iguais ao solicitado;
<code>IGUAL</code>	Para valores iguais ao solicitado;
<code>MENOR_OU_IGUAL</code>	Para valores menores ou iguais ao solicitado;
<code>MENOR</code>	Para valores menores do que o solicitado;
<code>PROXIMO</code>	Para valores próximos ao solicitado. Apenas para valores do tipo string.

5.4.2 Funções

5.4.2.1 `compararProximo()`

```
int compararProximo (
    Texto primeiro,
    Texto segundo )
```

Verifica se o segundo argumento é substring do primeiro. Retorna 1 em caso de sucesso e 0, caso contrário.

Parâmetros

<i>primeiro</i>	Estrutura de texto.
<i>segundo</i>	Estrutura de texto.

Retorna

1 caso de sucesso e 0, caso contrário.

5.4.2.2 compararValores()

```
int compararValores (
    Texto primeiro,
    Texto segundo,
    char tipo )
```

Compara valores de duas estruturas, convertendo-os nos tipos 'c', 'i', 'f', 'd', 's', retornando um valor negativo se o primeiro valor for menor do que o segundo, 0 se os valores forem iguais ou um valor positivo se o primeiro valor for maior do que o segundo.

Observação: Para o caso em que o tipo de dado for 'f', a implementação será idêntica ao caso em que o tipo de dado for 'd'.

Parâmetros

<i>primeiro</i>	Estrutura contendo os dados do primeiro valor
<i>segundo</i>	Estrutura contendo os dados do segundo valor
<i>tipo</i>	Tipo de dado referente aos valores a serem comparados

Retorna

um valor negativo se o primeiro valor for menor do que o segundo, 0 se os valores forem iguais ou um valor positivo se o primeiro valor for maior do que o segundo.

5.4.2.3 deveExibir()

```
int deveExibir (
    Texto valor,
    Texto comparar,
    Regra regra,
    char tipo )
```

Verifica a linha satisfaz a regra informada.

Parâmetros

<i>valor</i>	Estrutura de texto contendo o valor solicitado pelo usuário.
<i>comparar</i>	Estrutura de texto contendo os valores a serem comparados.
<i>regra</i>	Opção de pesquisa escolhida pelo usuário.
<i>tipo</i>	Tipo de dado

Retorna

1 em caso de sucesso e 0, caso contrário.

5.4.2.4 pesquisa_exibir()

```
void pesquisa_exibir (
    Tabela tabela,
    int indice,
    Texto valor,
    Regra regra )
```

Exibe na tela os resultados da pesquisa de valores.

Parâmetros

<i>tabela</i>	Estrutura contendo os dados da tabela.
<i>indice</i>	posição da coluna
<i>valor</i>	Estrutura contendo os dados do valor a ser pesquisado
<i>regra</i>	Estrutura referente à opção de pesquisa escolhida pelo usuário.

5.4.2.5 pesquisar()

```
void pesquisar (
    Tabela tabela,
    int indice )
```

Exibe na tela o menu com as opções de pesquisa, faz a leitura da escolha do usuário e do valor a ser comparado e exibe os resultados da pesquisa.

Parâmetros

<i>tabela</i>	Estrutura contendo os dados da tabela
<i>indice</i>	Posição da coluna

5.5 Referência do Arquivo Pesquisa.h

```
#include <string.h>
#include "Tabela.h"
```

Funções

- void [pesquisar](#) (Tabela tabela, int indice)

5.5.1 Funções

5.5.1.1 pesquisar()

```
void pesquisar (
    Tabela tabela,
    int indice )
```

Exibe na tela o menu com as opções de pesquisa, faz a leitura da escolha do usuário e do valor a ser comparado e exibe os resultados da pesquisa.

Parâmetros

<i>tabela</i>	Estrutura contendo os dados da tabela
<i>indice</i>	Posição da coluna

5.6 Pesquisa.h

[Ir para a documentação desse arquivo.](#)

```
00001
00002 #ifndef PESQUISA_H
00003 #define PESQUISA_H
00004
00005 #include <string.h>
00006 #include "Tabela.h"
00007
```

```
00008 void pesquisar(Tabela tabela, int indice);
00009
00010 #endif
```

5.7 Referência do Arquivo principal.c

```
#include <stdio.h>
#include <stdlib.h>
#include "Dados.h"
#include "Tabela.h"
#include "Pesquisa.h"
#include "TabelaGeral.h"
```

Enumerações

- enum **Opcoes** {
CRIAR_TABELA = 1 , **LISTAR_TABELA** = 2 , **CRIAR_LINHA** = 3 , **LISTAR_DADOS** = 4 ,
PESQUISAR_VALOR = 5 , **APAGAR_TUPLA** = 6 , **APAGAR_TABELA** = 7 , **SAIR** = 8 ,
LIMPAR = 0 }

Funções

- void **menu** ()
- int **verificarNomesIguais** (int n, **Texto** linhaASerInserida[])
- void **criarTabela** ()
- void **exibirDados** ()
- Texto** **leituraDaChavePrimaria** ()
- Texto** * **leituraDeDados** (int n, **ColunaDaTabela** colunas[])
- int **verificarExistenciaDaChave** (**Tabela** tabela, **Texto** linhaASerInserida[])
- void **adicionarDados** ()
- void **apagarTabela** ()
- void **apagarTupla** ()
- void **pesquisarValores** ()
- int **main** (void)

5.7.1 Enumerações

5.7.1.1 Opcoes

enum **Opcoes**

Representa as opções de escolha do menu do programa.

Enumeradores

CRIAR_TABELA	
LISTAR_TABELA	
CRIAR_LINHA	
LISTAR_DADOS	
PESQUISAR_VALOR	
APAGAR_TUPLA	
APAGAR_TABELA	
SAIR	
LIMPAR	

5.7.2 Funções

5.7.2.1 adicionarDados()

```
void adicionarDados ( )
```

Representa a opção do menu de adicionar linha em uma tabela.

Faz a leitura do nome da tabela e dos dados a serem adicionados e, em seguida, atualiza o arquivo da tabela, inserindo uma linha com os dados recentes.

5.7.2.2 apagarTabela()

```
void apagarTabela ( )
```

Representa a opção do menu de apagar uma tabela.

Faz a leitura do nome da tabela e a apaga do disco rígido, atualizando o arquivo 'tabela geral.data', que contém a quantidade de tabelas e seus nomes.

5.7.2.3 apagarTupla()

```
void apagarTupla ( )
```

Representa a opção do menu de apagar uma linha de uma tabela.

Faz a leitura do nome da tabela e da chave desejada e apaga sua linha, atualizando o arquivo da tabela.

5.7.2.4 criarTabela()

```
void criarTabela ( )
```

Representa a opção do menu de criar uma tabela.

Realiza a leitura do nome da tabela, verificando se este é válido, da quantidade de colunas da tabela, dos nomes das colunas (incluindo a chave primária) e seus tipos, criando um arquivo contendo essas informações e outro contendo a quantidade de tabelas e seus nomes (tabela geral.data).

5.7.2.5 exibirDados()

```
void exibirDados ( )
```

Representa a opção do menu de exibir os dados de uma tabela.

Faz a leitura do nome da tabela, abre seu arquivo correspondente e exibe na tela seu cabeçalho com seus dados.

5.7.2.6 leituraDaChavePrimaria()

```
Texto leituraDaChavePrimaria ( )
```

Faz a leitura da chave primária retornando uma estrutura contendo suas informações.

Retorna

Estrutura com os dados lidos.

5.7.2.7 leituraDeDados()

```
Texto * leituraDeDados (
    int n,
    ColunaDaTabela colunas[] )
```

Faz as leituras dos dados da linha a ser inserida, retornando uma estrutura com os dados lidos.

Observação: Insere 'NULL' caso o usuário não queira adicionar um dado.

Parâmetros

<i>n</i>	quantidade de colunas
<i>colunas</i>	Estrutura contendo os dados das colunas

Retorna

Uma estrutura com os dados lidos

5.7.2.8 main()

```
int main (
    void )
```

Função principal.

Retorna

Resultado da execução do programa. 0 indica ausência de erros.

Instruções de acordo com a escolha do usuário.

5.7.2.9 menu()

```
void menu ( )
```

Exibe na tela as opções do menu.

5.7.2.10 pesquisarValores()

```
void pesquisarValores ( )
```

Representa a opção do menu de pesquisar um valor em uma tabela.

Faz a leitura do nome da tabela e exibe as colunas disponíveis. Depois, faz a leitura do valor a ser pesquisado, exibindo as opções de pesquisa para, posteriormente, exibir os resultados da pesquisa.

5.7.2.11 verificarExistenciaDaChave()

```
int verificarExistenciaDaChave (
    Tabela tabela,
    Texto linhaASerInserida[ ] )
```

Verifica se já existe uma chave com valor igual ao requerido, retornando 1 em caso de sucesso e 0, caso contrário.

Parâmetros

<i>tabela</i>	Estrutura com os dados da tabela.
<i>linhaASerInserida</i>	Estrutura com o dado a ser verificado

Retorna

1 em caso de sucesso e 0, caso contrário

5.7.2.12 verificarNomesIguais()

```
int verificarNomesIguais (
    int n,
    Texto linhaASerInserida[ ] )
```

Verifica se já existe um texto registrado igual ao requerido.

Recebe um vetor de textos e verifica se existe dois textos iguais nesse vetor. Retorna 0 se não existirem valores iguais ou, caso contrário, a posição de maior valor entre os dois textos comparados, ou seja, nunca retorna zero em caso verdadeiro.

Parâmetros

<i>n</i>	quantidade de textos a serem comparadas.
<i>linhaASerInserida</i>	vetor com todos os textos.

Retorna

retorna 0 se não existir dois textos iguais no vetor, caso contrário, retorna valor maior que zero.

5.8 Referência do Arquivo README.md

5.9 Referência do Arquivo Tabela.c

```
#include "Tabela.h"
```

Funções

- [Tabela](#) [tabela_criar](#) ([Texto](#) nome, int quantidadeDeColunas, [Texto](#) nomesDasColunas[], char tiposDasColunas[])
- void [tabela_gravar](#) ([Tabela](#) tabela)
- [Tabela](#) [tabela_recuperar](#) ([Texto](#) nome)
- void [tabela_liberar](#) ([Tabela](#) tabela)
- void [tabela_exibirCabecalho](#) ([Tabela](#) tabela)
- void [tabela_exibirDados](#) ([Tabela](#) tabela)
- [Tabela](#) [tabela_adicionarLinha](#) ([Tabela](#) tabela, [Texto](#) dados[])
- int [coluna_buscarValor](#) ([ColunaDaTabela](#) coluna, [Texto](#) valor)
- int [tabela_removeChave](#) ([Tabela](#) tabela, [Texto](#) chave)

5.9.1 Funções

5.9.1.1 coluna_buscarValor()

```
int coluna_buscarValor (
    ColunaDaTabela coluna,
    Texto valor )
```

Verifica se um dado existe nas colunas de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados da coluna de uma tabela e outra contendo os dados de um valor. Depois, compara cada dado da coluna com esse valor, retornando a posição do dado valor na coluna em caso de sucesso (quando são iguais) e -1, caso contrário.

Parâmetros

<i>coluna</i>	Estrutura de texto contendo os dados da tabela.
<i>valor</i>	Estrutura de texto contendo os dados de um valor.

Retorna

-1 se o valor não estiver presente. Caso contrário, retorna a posição do valor na coluna.

5.9.1.2 tabela_adicionarLinha()

```
Tabela tabela_adicionarLinha (
    Tabela tabela,
    Texto dados[] )
```

Adiciona uma linha para cada coluna de uma tabela.

Recebe como parâmetro uma estrutura texto contendo os dados a serem adicionados e uma estrutura contendo os dados da tabela. Em seguida, atualiza a estrutura [Tabela](#) com os dados da estrutura [Texto](#) retornando-a no final.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
<i>dados</i>	Estrutura de string contendo os dados a serem adicionados nas colunas da tabela.

Retorna

Estrutura [Tabela](#) contendo os dados da tabela atualizados.

5.9.1.3 tabela_criar()

```
Tabela tabela_criar (
    Texto nome,
    int quantidadeDeColunas,
    Texto nomesDasColunas[],
    char tiposDasColunas[] )
```

Cria uma tabela.

Recebe como argumentos o nome da tabela, a quantidade de colunas que a tabela terá, um vetor com os nomes das colunas e outro com os tipos de dados associados à elas. Retorna uma estrutura preenchida com essas informações.

Parâmetros

<i>nome</i>	Estrutura contendo os dados do nome da tabela.
<i>quantidadeDeColunas</i>	A quantidade de colunas que a tabela terá.
<i>nomesDasColunas</i>	Vetor de string contendo os nomes das colunas.
<i>tiposDasColunas</i>	Vetor de string contendo os tipos de dados associados às colunas.

Retorna

Estrutura preenchidas com as informações recebidas.

5.9.1.4 tabela_exibirCabecalho()

```
void tabela_exibirCabecalho (
    Tabela tabela )
```

Exibe na tela o cabeçalho de uma tabela.

Recebe como parâmetro uma estrutura contendo o nome de uma tabela, a quantidade de linhas e colunas, os nomes e os tipos das colunas e exibe essas informações na tela.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.9.1.5 tabela_exibirDados()

```
void tabela_exibirDados (
    Tabela tabela )
```

Exibe na tela os dados das colunas de uma tabela.

Recebe como parâmetro uma estrutura contendo os dados de uma tabela e exibe na tela os dados de cada coluna.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.9.1.6 tabela_gravar()

```
void tabela_gravar (
    Tabela tabela )
```

Cria um arquivo referente a uma tabela.

Recebe como argumento uma estrutura contendo informações de uma tabela adquiridas previamente em outra função e cria um arquivo para essa tabela, repassando as informações contidas na estrutura para esse arquivo. Observação: o arquivo será nomeado pelo nome da tabela.

Parâmetros

<i>tabela</i>	Estrutura contendo as informações de uma tabela.
---------------	--

5.9.1.7 `tabela_liberar()`

```
void tabela_liberar (  
    Tabela tabela )
```

Liberação da memória alocada dinamicamente para a estrutura tabela.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.9.1.8 `tabela_recuperar()`

```
Tabela tabela_recuperar (  
    Texto nome )
```

Faz a leitura dos dados de um arquivo referente a uma tabela.

Recebe como argumento uma estrutura de texto contendo os dados do nome de uma tabela, abre o arquivo correspondente à essa tabela, em modo leitura e faz a leitura de seus dados, retornando uma estrutura contendo os dados lidos.

Parâmetros

<i>nome</i>	Estrutura de texto contendo os dados do nome da tabela.
-------------	---

Retorna

Uma estrutura contendo os dados lidos.

5.9.1.9 `tabela_removeChave()`

```
int tabela_removeChave (  
    Tabela tabela,  
    Texto chave )
```

Remove uma linha (tupla) de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados de uma tabela e outra contendo os dados de um valor correspondente à chave da linha a ser removida. Depois, localiza sua posição na coluna, caso ela exista e atualiza o arquivo da tabela com os dados restantes, excluindo-se os referentes à essa posição. Retorna 1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave solicitada não exista.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
<i>chave</i>	Estrutura de texto contendo os dados da chave referente à linha a ser removida

Retorna

1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave não exista.

5.10 Referência do Arquivo Tabela.h

```
#include <stdio.h>
#include <string.h>
#include "Dados.h"
```

Estruturas de Dados

- struct [ColunaDaTabela](#)
- struct [Tabela](#)

Funções

- [Tabela](#) [tabela_criar](#) ([Texto](#) nome, int quantidadeDeColunas, [Texto](#) nomeDasColunas[], char tipoDasColunas[])
- void [tabela_gravar](#) ([Tabela](#) tabela)
- [Tabela](#) [tabela_recuperar](#) ([Texto](#) nome)
- void [tabela_liberar](#) ([Tabela](#) tabela)
- void [tabela_exibirCabecalho](#) ([Tabela](#) tabela)
- void [tabela_exibirDados](#) ([Tabela](#) tabela)
- int [tabela_removeChave](#) ([Tabela](#) tabela, [Texto](#) chave)
- [Tabela](#) [tabela_adicionarLinha](#) ([Tabela](#) tabela, [Texto](#) dados[])
- int [coluna_buscarValor](#) ([ColunaDaTabela](#) coluna, [Texto](#) valor)

5.10.1 Funções

5.10.1.1 coluna_buscarValor()

```
int coluna_buscarValor (
    ColunaDaTabela coluna,
    Texto valor )
```

Verifica se um dado existe nas colunas de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados da coluna de uma tabela e outra contendo os dados de um valor. Depois, compara cada dado da coluna com esse valor, retornando a posição do dado valor na coluna em caso de sucesso (quando são iguais) e -1, caso contrário.

Parâmetros

<i>coluna</i>	Estrutura de texto contendo os dados da tabela.
<i>valor</i>	Estrutura de texto contendo os dados de um valor.

Retorna

-1 se o valor não estiver presente. Caso contrário, retorna a posição do valor na coluna.

5.10.1.2 tabela_adicionarLinha()

```
Tabela tabela_adicionarLinha (
    Tabela tabela,
    Texto dados[] )
```

Adiciona uma linha para cada coluna de uma tabela.

Recebe como parâmetro uma estrutura texto contendo os dados a serem adicionados e uma estrutura contendo os dados da tabela. Em seguida, atualiza a estrutura [Tabela](#) com os dados da estrutura [Texto](#) retornando-a no final.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
<i>dados</i>	Estrutura de string contendo os dados a serem adicionados nas colunas da tabela.

Retorna

Estrutura [Tabela](#) contendo os dados da tabela atualizados.

5.10.1.3 tabela_criar()

```
Tabela tabela_criar (
    Texto nome,
    int quantidadeDeColunas,
    Texto nomesDasColunas[ ],
    char tiposDasColunas[ ] )
```

Cria uma tabela.

Recebe como argumentos o nome da tabela, a quantidade de colunas que a tabela terá, um vetor com os nomes das colunas e outro com os tipos de dados associados à elas. Retorna uma estrutura preenchida com essas informações.

Parâmetros

<i>nome</i>	Estrutura contendo os dados do nome da tabela.
<i>quantidadeDeColunas</i>	A quantidade de colunas que a tabela terá.
<i>nomesDasColunas</i>	Vetor de string contendo os nomes das colunas.
<i>tiposDasColunas</i>	Vetor de string contendo os tipos de dados associados às colunas.

Retorna

Estrutura preenchidas com as informações recebidas.

5.10.1.4 tabela_exibirCabecalho()

```
void tabela_exibirCabecalho (
    Tabela tabela )
```

Exibe na tela o cabeçalho de uma tabela.

Recebe como parâmetro uma estrutura contendo o nome de uma tabela, a quantidade de linhas e colunas, os nomes e os tipos das colunas e exibe essas informações na tela.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.10.1.5 tabela_exibirDados()

```
void tabela_exibirDados (
    Tabela tabela )
```

Exibe na tela os dados das colunas de uma tabela.

Recebe como parâmetro uma estrutura contendo os dados de uma tabela e exibe na tela os dados de cada coluna.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.10.1.6 tabela_gravar()

```
void tabela_gravar (
    Tabela tabela )
```

Cria um arquivo referente a uma tabela.

Recebe como argumento uma estrutura contendo informações de uma tabela adquiridas previamente em outra função e cria um arquivo para essa tabela, repassando as informações contidas na estrutura para esse arquivo.

Observação: o arquivo será nomeado pelo nome da tabela.

Parâmetros

<i>tabela</i>	Estrutura contendo as informações de uma tabela.
---------------	--

5.10.1.7 tabela_liberar()

```
void tabela_liberar (
    Tabela tabela )
```

Liberação da memória alocada dinamicamente para a estrutura tabela.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
---------------	---

5.10.1.8 tabela_recuperar()

```
Tabela tabela_recuperar (
    Texto nome )
```

Faz a leitura dos dados de um arquivo referente a uma tabela.

Recebe como argumento uma estrutura de texto contendo os dados do nome de uma tabela, abre o arquivo correspondente à essa tabela, em modo leitura e faz a leitura de seus dados, retornando uma estrutura contendo os dados lidos.

Parâmetros

<i>nome</i>	Estrutura de texto contendo os dados do nome da tabela.
-------------	---

Retorna

Uma estrutura contendo os dados lidos.

5.10.1.9 tabela_removeChave()

```
int tabela_removeChave (
    Tabela tabela,
    Texto chave )
```

Remove uma linha (tupla) de uma tabela.

Recebe como parâmetros uma estrutura contendo os dados de uma tabela e outra contendo os dados de um valor correspondente à chave da linha a ser removida. Depois, localiza sua posição na coluna, caso ela exista e atualiza o arquivo da tabela com os dados restantes, excluindo-se os referentes à essa posição. Retorna 1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave solicitada não exista.

Parâmetros

<i>tabela</i>	Estrutura de texto contendo os dados da tabela.
<i>chave</i>	Estrutura de texto contendo os dados da chave referente à linha a ser removida

Retorna

1 caso a remoção da linha seja realizada com sucesso e 0 caso a chave não exista.

5.11 Tabela.h

[Ir para a documentação desse arquivo.](#)

```
00001
00002 #ifndef TABELA_H
00003 #define TABELA_H
00004
00005 #include <stdio.h>
00006 #include <string.h>
00007
00008 #include "Dados.h"
00009
00015 typedef struct {
00016     Texto nome;
00017     char tipo;
00018     int quantidadeDeLinhas;
00019     Texto * dados;
00020 } ColunaDaTabela;
00021
00027 typedef struct {
00028     Texto nome;
00029     int quantidadeDeColunas;
00030     int quantidadeDeLinhas;
00031     ColunaDaTabela * colunas;
00032 } Tabela;
00033
00034 Tabela tabela_criar(Texto nome, int quantidadeDeColunas, Texto nomeDasColunas[], char
    tipoDasColunas[]);
00035 void tabela_gravar(Tabela tabela);
00036 Tabela tabela_recuperar(Texto nome);
00037 void tabela_liberar(Tabela tabela);
00038 void tabela_exibirCabecalho(Tabela tabela);
00039 void tabela_exibirDados(Tabela tabela);
00040 int tabela_removeChave(Tabela tabela, Texto chave);
00041
00042 Tabela tabela_adicionarLinha(Tabela tabela, Texto dados []);
00043
00044 // retorna -1 se o valor não estiver na coluna, em caso contrário, retorna a posição em que ele se
    encontra
00045 int coluna_buscarValor(ColunaDaTabela coluna, Texto valor);
00046
00047 #endif
```

5.12 Referência do Arquivo TabelaGeral.c

```
#include <stdio.h>
#include <stdlib.h>
#include "Dados.h"
#include "TabelaGeral.h"
```

Definições e Macros

- #define NOME_DA_TABELA_GERAL "tabela geral.data"

Funções

- Texto * tabelaGeral_recuperarDados (FILE *arquivo, int *n)
- void tabelaGeral_gravarDados (FILE *arquivo, int quantidadeDeTabelas, Texto nomesDasTabelas[])
- void tabelaGeral_adicionarTabela (Texto nomeDaNovaTabela)
- int tabelaGeral_contemTabela (Texto nome)
- void tabelaGeral_limpar ()
- int tabelaGeral_apagar (Texto excluindo)
- void tabelaGeral_listar ()

5.12.1 Definições e macros

5.12.1.1 NOME_DA_TABELA_GERAL

```
#define NOME_DA_TABELA_GERAL "tabela geral.data"
```

5.12.2 Funções

5.12.2.1 tabelaGeral_adicionarTabela()

```
void tabelaGeral_adicionarTabela (
    Texto nomeDaNovaTabela )
```

Cria ou atualiza um arquivo contendo a quantidade de tabelas e seus respectivos nomes, sempre que uma nova tabela é criada.

Caso o arquivo não exista, um novo é criado. Caso contrário, seus dados são atualizados.

Parâmetros

<i>nomeDaNovaTabela</i>	Estrutura com o nome da nova tabela criada previamente.
-------------------------	---

5.12.2.2 tabelaGeral_apagar()

```
int tabelaGeral_apagar (
    Texto excluindo )
```

Apaga do disco rígido uma tabela específica e atualiza o arquivo que lista os nomes das tabelas existentes.

Recebe como argumento uma estrutura contendo o nome da tabela a ser apagada e atualiza os dados do arquivo que contém os nomes de todas as tabelas existentes, excluindo-se o nome da tabela que foi apagada.

Parâmetros

<i>excluindo</i>	Estrutura contendo os dados a serem deletados.
------------------	--

Retorna

1 se a tabela existir e a remoção for realizada e 0, caso a tabela não exista.

5.12.2.3 tabelaGeral_contemTabela()

```
int tabelaGeral_contemTabela (
    Texto nome )
```

Recebe uma estrutura de texto que corresponde a um nome e verifica se esse nome já existe dentro de um arquivo. Abre um arquivo em modo leitura, realiza a leitura de seus dados e compara se o dado recebido é igual a algum dado lido. Retorna 1 em caso de sucesso e 0, caso contrário.

Parâmetros

<i>nome</i>	Estrutura de Texto .
-------------	--------------------------------------

Retorna

1 se o nome já existir e 0, caso contrário.

5.12.2.4 tabelaGeral_gravarDados()

```
void tabelaGeral_gravarDados (
    FILE * arquivo,
```

```
int quantidadeDeTabelas,
Texto nomesDasTabelas[ ] )
```

Grava no arquivo os dados atualizados sobre a quantidade de tabelas e seus nomes.

Parâmetros

<i>arquivo</i>	Arquivo a ter dados gravados. Deve estar aberto no modo de escrita.
<i>quantidadeDeTabelas</i>	Quantidade atualizada.
<i>nomesDasTabelas</i>	Texto com os dados atualizados.

5.12.2.5 tabelaGeral_limpar()

```
void tabelaGeral_limpar ( )
```

Apaga do disco rígido todos os arquivos salvos.

5.12.2.6 tabelaGeral_listar()

```
void tabelaGeral_listar ( )
```

Lista os nomes das tabelas existentes.

Faz a leitura dos dados do arquivo 'tabela geral.data' e exibe na tela os dados correspondentes aos nomes das tabelas.

5.12.2.7 tabelaGeral_recuperarDados()

```
Texto * tabelaGeral_recuperarDados (
    FILE * arquivo,
    int * n )
```

Faz a leitura dos dados de um arquivo aberto em modo de leitura, atribuindo os dados lidos em uma estrutura de strings. Os dados do arquivo estão organizados na seguinte sequência: o primeiro dado corresponde à quantidade de tabelas existentes, enquanto que os dados restantes correspondem aos nomes das tabelas, representados pela estrutura [Texto](#).

Parâmetros

<i>arquivo</i>	Arquivo a ser lido.
<i>n</i>	Ponteiro para quantidade de tabelas.

Retorna

Uma estrutura de [Texto](#) contendo os dados lidos, além de passar por referência o dado correspondente à quantidade de tabelas.

5.13 Referência do Arquivo TabelaGeral.h

```
#include <string.h>
#include "Dados.h"
#include "Tabela.h"
```

Funções

- int [tabelaGeral_contemTabela](#) ([Texto](#) nome)
- void [tabelaGeral_adicionarTabela](#) ([Texto](#) nome)
- void [tabelaGeral_limpar](#) ()
- int [tabelaGeral_apagar](#) ([Texto](#) nomeDaTabela)
- void [tabelaGeral_listar](#) ()

5.13.1 Funções

5.13.1.1 tabelaGeral_adicionarTabela()

```
void tabelaGeral_adicionarTabela (
    Texto nomeDaNovaTabela )
```

Cria ou atualiza um arquivo contendo a quantidade de tabelas e seus respectivos nomes, sempre que uma nova tabela é criada.

Caso o arquivo não exista, um novo é criado. Caso contrário, seus dados são atualizados.

Parâmetros

<i>nomeDaNovaTabela</i>	Estrutura com o nome da nova tabela criada previamente.
-------------------------	---

5.13.1.2 tabelaGeral_apagar()

```
int tabelaGeral_apagar (
    Texto excluindo )
```

Apaga do disco rígido uma tabela específica e atualiza o arquivo que lista os nomes das tabelas existentes.

Recebe como argumento uma estrutura contendo o nome da tabela a ser apagada e atualiza os dados do arquivo que contém os nomes de todas as tabelas existentes, excluindo-se o nome da tabela que foi apagada.

Parâmetros

<i>excluindo</i>	Estrutura contendo os dados a serem deletados.
------------------	--

Retorna

1 se a tabela existir e a remoção for realizada e 0, caso a tabela não exista.

5.13.1.3 tabelaGeral_contemTabela()

```
int tabelaGeral_contemTabela (
    Texto nome )
```

Recebe uma estrutura de texto que corresponde a um nome e verifica se esse nome já existe dentro de um arquivo. Abre um arquivo em modo leitura, realiza a leitura de seus dados e compara se o dado recebido é igual a algum dado lido. Retorna 1 em caso de sucesso e 0, caso contrário.

Parâmetros

<i>nome</i>	Estrutura de Texto .
-------------	--------------------------------------

Retorna

1 se o nome já existir e 0, caso contrário.

5.13.1.4 tabelaGeral_limpar()

```
void tabelaGeral_limpar ( )
```

Apaga do disco rígido todos os arquivos salvos.

5.13.1.5 tabelaGeral_listar()

```
void tabelaGeral_listar ( )
```

Lista os nomes das tabelas existentes.

Faz a leitura dos dados do arquivo 'tabela geral.data' e exibe na tela os dados correspondentes aos nomes das tabelas.

5.14 TabelaGeral.h

[Ir para a documentação desse arquivo.](#)

```
00001
00002 #ifndef TABELA_GERAL_H
00003 #define TABELA_GERAL_H
00004
00005 #include <string.h>
00006
00007 #include "Dados.h"
00008 #include "Tabela.h"
00009
00010 int tabelaGeral_contemTabela(Texto nome);
00011 void tabelaGeral_adicionarTabela(Texto nome);
00012 void tabelaGeral_limpar();
00013 int tabelaGeral_apagar(Texto nomeDaTabela);
00014 void tabelaGeral_listar();
00015
00016 #endif
```


Índice Remissivo

adicionarDados
 principal.c, 21
APAGAR_TABELA
 principal.c, 20
APAGAR_TUPLA
 principal.c, 20
apagarTabela
 principal.c, 21
apagarTupla
 principal.c, 21

capacidade
 Texto, 9
coluna_buscarValor
 Tabela.c, 23
 Tabela.h, 26
ColunaDaTabela, 7
 dados, 7
 nome, 7
 quantidadeDeLinhas, 7
 tipo, 7
colunas
 Tabela, 8
compararProximo
 Pesquisa.c, 17
compararValores
 Pesquisa.c, 18
CRIAR_LINHA
 principal.c, 20
CRIAR_TABELA
 principal.c, 20
criarTabela
 principal.c, 21

dados
 ColunaDaTabela, 7
Dados.c, 11
 ehDigito, 11
 ehMaiuscula, 11
 ehMinuscula, 12
 ehSimbolo, 12
 texto_gravar, 12
 texto_lerTerminalEmLoop, 12
 texto_lerTipoEmLoop, 13
 texto_lerValorComTipo, 13
 texto_lerValorDoTipoString, 13
 texto_lerValorPositivoEmLoop, 13
 texto_recuperar, 14
 verificaNomeDaTabelaValido, 14
Dados.h, 14
 texto_gravar, 15
 texto_lerTerminalEmLoop, 15
 texto_lerTipoEmLoop, 15
 texto_lerValorComTipo, 15
 texto_lerValorPositivoEmLoop, 16
 texto_recuperar, 16
deveExibir
 Pesquisa.c, 18

ehDigito
 Dados.c, 11
ehMaiuscula
 Dados.c, 11
ehMinuscula
 Dados.c, 12
ehSimbolo
 Dados.c, 12
exibirDados
 principal.c, 21

IGUAL
 Pesquisa.c, 17

leituraDaChavePrimaria
 principal.c, 21
leituraDeDados
 principal.c, 21
LIMPAR
 principal.c, 20
LISTAR_DADOS
 principal.c, 20
LISTAR_TABELA
 principal.c, 20

main
 principal.c, 22
MAIOR
 Pesquisa.c, 17
MAIOR_OU_IGUAL
 Pesquisa.c, 17
MENOR
 Pesquisa.c, 17
MENOR_OU_IGUAL
 Pesquisa.c, 17
menu
 principal.c, 22

nome
 ColunaDaTabela, 7
 Tabela, 8

NOME_DA_TABELA_GERAL
TabelaGeral.c, 30

Opcoes
principal.c, 20

Pesquisa.c, 17
compararProximo, 17
compararValores, 18
deveExibir, 18
IGUAL, 17
MAIOR, 17
MAIOR_OU_IGUAL, 17
MENOR, 17
MENOR_OU_IGUAL, 17
pesquisa_exibir, 18
pesquisar, 19
PROXIMO, 17
Regra, 17

Pesquisa.h, 19
pesquisar, 19

pesquisa_exibir
Pesquisa.c, 18

pesquisar
Pesquisa.c, 19
Pesquisa.h, 19

PESQUISAR_VALOR
principal.c, 20

pesquisarValores
principal.c, 22

principal.c, 20
adicionarDados, 21
APAGAR_TABELA, 20
APAGAR_TUPLA, 20
apagarTabela, 21
apagarTupla, 21
CRIAR_LINHA, 20
CRIAR_TABELA, 20
criarTabela, 21
exibirDados, 21
leituraDaChavePrimaria, 21
leituraDeDados, 21
LIMPAR, 20
LISTAR_DADOS, 20
LISTAR_TABELA, 20
main, 22
menu, 22
Opcoes, 20
PESQUISAR_VALOR, 20
pesquisarValores, 22
SAIR, 20
verificarExistenciaDaChave, 22
verificarNomesIguais, 22

PROXIMO
Pesquisa.c, 17

quantidadeDeColunas
Tabela, 8

quantidadeDeLinhas

ColunaDaTabela, 7
Tabela, 8

README.md, 23
Regra
Pesquisa.c, 17

SAIR
principal.c, 20
SGBD ITP, 1

Tabela, 8
colunas, 8
nome, 8
quantidadeDeColunas, 8
quantidadeDeLinhas, 8

Tabela.c, 23
coluna_buscarValor, 23
tabela_adicionarLinha, 23
tabela_criar, 24
tabela_exibirCabecalho, 24
tabela_exibirDados, 24
tabela_gravar, 24
tabela_liberar, 25
tabela_recuperar, 25
tabela_removeChave, 25

Tabela.h, 26
coluna_buscarValor, 26
tabela_adicionarLinha, 26
tabela_criar, 27
tabela_exibirCabecalho, 27
tabela_exibirDados, 27
tabela_gravar, 28
tabela_liberar, 28
tabela_recuperar, 28
tabela_removeChave, 28

tabela_adicionarLinha
Tabela.c, 23
Tabela.h, 26

tabela_criar
Tabela.c, 24
Tabela.h, 27

tabela_exibirCabecalho
Tabela.c, 24
Tabela.h, 27

tabela_exibirDados
Tabela.c, 24
Tabela.h, 27

tabela_gravar
Tabela.c, 24
Tabela.h, 28

tabela_liberar
Tabela.c, 25
Tabela.h, 28

tabela_recuperar
Tabela.c, 25
Tabela.h, 28

tabela_removeChave
Tabela.c, 25

- Tabela.h, [28](#)
- TabelaGeral.c, [29](#)
 - NOME_DA_TABELA_GERAL, [30](#)
 - tabelaGeral_adicionarTabela, [30](#)
 - tabelaGeral_apagar, [30](#)
 - tabelaGeral_contemTabela, [30](#)
 - tabelaGeral_gravarDados, [30](#)
 - tabelaGeral_limpar, [31](#)
 - tabelaGeral_listar, [31](#)
 - tabelaGeral_recuperarDados, [31](#)
- TabelaGeral.h, [31](#)
 - tabelaGeral_adicionarTabela, [32](#)
 - tabelaGeral_apagar, [32](#)
 - tabelaGeral_contemTabela, [32](#)
 - tabelaGeral_limpar, [32](#)
 - tabelaGeral_listar, [32](#)
- tabelaGeral_adicionarTabela
 - TabelaGeral.c, [30](#)
 - TabelaGeral.h, [32](#)
- tabelaGeral_apagar
 - TabelaGeral.c, [30](#)
 - TabelaGeral.h, [32](#)
- tabelaGeral_contemTabela
 - TabelaGeral.c, [30](#)
 - TabelaGeral.h, [32](#)
- tabelaGeral_gravarDados
 - TabelaGeral.c, [30](#)
- tabelaGeral_limpar
 - TabelaGeral.c, [31](#)
 - TabelaGeral.h, [32](#)
- tabelaGeral_listar
 - TabelaGeral.c, [31](#)
 - TabelaGeral.h, [32](#)
- tabelaGeral_recuperarDados
 - TabelaGeral.c, [31](#)
- tamanho
 - Texto, [9](#)
- Texto, [8](#)
 - capacidade, [9](#)
 - tamanho, [9](#)
 - vetor, [9](#)
- texto_gravar
 - Dados.c, [12](#)
 - Dados.h, [15](#)
- texto_lerTerminalEmLoop
 - Dados.c, [12](#)
 - Dados.h, [15](#)
- texto_lerTipoEmLoop
 - Dados.c, [13](#)
 - Dados.h, [15](#)
- texto_lerValorComTipo
 - Dados.c, [13](#)
 - Dados.h, [15](#)
- texto_lerValorDoTipoString
 - Dados.c, [13](#)
- texto_lerValorPositivoEmLoop
 - Dados.c, [13](#)
 - Dados.h, [16](#)
- texto_recuperar
 - Dados.c, [14](#)
 - Dados.h, [16](#)
- tipo
 - ColunaDaTabela, [7](#)
- verificaNomeDaTabelaValido
 - Dados.c, [14](#)
- verificarExistenciaDaChave
 - principal.c, [22](#)
- verificarNomesIguais
 - principal.c, [22](#)
- vetor
 - Texto, [9](#)